

ENTERPRISE KUBERNETES MASTERY

AI Platform Engineering Handbook

PART XIII EMERGING STANDARDS AND ECOSYSTEM TRENDS

MCP, A2A, AI Gateways, OpenTelemetry GenAI, Ecosystem Convergence

Volume 13 of 16 Advanced Series | Edition 2025-2026

CHAPTER 1

Model Context Protocol (MCP) -- Ecosystem and Kubernetes Patterns

MCP (Anthropic, November 2024) has become the de facto standard for LLM-to-tool connectivity. By mid-2025, the ecosystem includes hundreds of MCP servers for: file systems, databases (PostgreSQL, SQLite, MongoDB, Redis), cloud platforms (AWS, GCP, Azure), developer tools (GitHub, GitLab, Jira, Linear, Slack), web browsing, code execution, and domain-specific enterprise integrations.

MCP Transport Evolution on Kubernetes

Transport	Status	Kubernetes Pattern	Scalability	Recommendation
stdio	GA (original)	Sidecar 1:1 with agent Pod	None (per-process)	Dev/testing only; not for production scale
SSE (HTTP Server-Sent Events)	GA	HTTP Deployment + Service	Good (multi-client)	Production for stateful tools
Streamable HTTP	GA (2025)	Deployment + KEDA ScaledObject	Excellent (stateless)	Preferred for all new production MCP servers
WebSocket	Experimental	Service + Gateway API	Good	Evaluate for real-time streaming tools

- * **MCP Server Operator:** Kubernetes Operator managing MCP server lifecycle, health checks, scaling, and auto-registration in the tool registry. Community proposals emerging; expect CNCF sandbox project by 2026.
- * **MCP over Service Mesh:** Running MCP server communications over Istio or Linkerd mTLS. Zero-code cryptographic authentication of MCP clients and servers. Eliminates application-level API key management for internal tool access.
- * **Gateway API for MCP routing:** Using HTTPRoute resources to route MCP requests to versioned backends, enabling blue-green and canary upgrades of MCP servers without client configuration changes.
- * **MCP ServerClass CRD:** Proposed standard CRD describing MCP server capabilities, schemas, and SLAs in a machine-readable format. Enables automated tool discovery by agents from the Kubernetes API.
- * **Federated MCP registries:** Enterprise MCP hub aggregating tools from multiple business units and vendors. Gateway provides unified endpoint; routes to appropriate backend MCP server based on tool name and namespace.

CHAPTER 2

Agent-to-Agent (A2A) Protocol -- Kubernetes Integration

A2A (Google, April 2025) defines a standard HTTP protocol for agent interoperability. Agents expose an Agent Card at /.well-known/agent.json describing capabilities and skills. Tasks flow between agents via structured HTTP requests with streaming support.

A2A Concept	Description	Kubernetes Implementation
Agent Card	JSON capability manifest at well-known URL	Service + Gateway HTTPRoute to /.well-known/agent.json
Task submission	POST /tasks with input and callback URL	Temporal workflow start or Argo Workflow submit
Task status	GET /tasks/{id}	Kubernetes CR status subresource (Agent CRD)
Streaming	SSE or WebSocket for partial results	Service + ingress-nginx SSE support
Authentication	OAuth 2.0 client credentials	SPIFFE SVID or Service Account OIDC token
Agent discovery	DNS + well-known URL convention	Kubernetes Service DNS + Gateway API
Error handling	Standard HTTP status codes + A2A error body	Temporal retry policies + circuit breaker

MCP and A2A: Complementary Standards

MCP and A2A serve complementary roles in enterprise agentic AI: MCP: Agent calls tools and reads resources (client -> server) A2A: Agents delegate tasks to other agents (peer -> peer) A sophisticated enterprise agent uses both simultaneously: 1. Receives task via A2A from orchestrating agent 2. Uses MCP to call web search, database, and code tools 3. Delegates sub-analysis to specialist agent via A2A 4. Returns structured result to orchestrator via A2A response

CHAPTER 3

AI Gateway Ecosystem and Inference Standards

Gateway	Primary Focus	K8s Deployment	2025 Differentiator
LiteLLM	LLM routing, 100+ providers	Deployment + Redis	Simplest; OpenAI-compatible; cost tracking
Kong AI Gateway	Enterprise API + AI plugins	Kong Operator	Enterprise RBAC, audit, rate limiting
Portkey	AI observability + guardrails	Deployment	Deep analytics, PII redaction, caching
NVIDIA NIM	Optimised model microservices	Helm + Operator	NVIDIA-tuned engines; TensorRT-LLM

Gateway	Primary Focus	K8s Deployment	2025 Differentiator
Envoy AI Gateway	High-performance L7 routing	Envoy filter (eBPF)	Sub-ms routing, eBPF integration
vLLM Production Server	Dedicated LLM inference	Deployment + KEDA	PagedAttention, 3-24x throughput

OpenAI API Compatibility as the De Facto Standard

The OpenAI REST API format (`/v1/chat/completions`, `/v1/embeddings`, `/v1/completions`) has emerged as the de facto standard for LLM inference APIs. vLLM, Ollama, LiteLLM, Anthropic (via proxy), and most open-source serving frameworks implement OpenAI compatibility. This means: changing LLM backends requires only changing the base URL and model name, not the application code. Design all AI applications against the OpenAI API spec and route via an AI gateway for backend flexibility.

CHAPTER 4

OpenTelemetry GenAI Semantic Conventions

The OTel GenAI SIG has standardised semantic conventions for LLM observability. These are stable as of OTel 1.26 and supported by all major vendors.

```
# Standard OTel span attributes for LLM calls: gen_ai.system: openai | anthropic | bedrock |
vertex_ai gen_ai.operation.name: chat | text_completion | embeddings gen_ai.request.model:
gpt-4o | claude-3-5-sonnet-20241022 gen_ai.request.max_tokens: 4096
gen_ai.request.temperature: 0.7 gen_ai.response.model: gpt-4o-2024-08-06 (actual deployed
version) gen_ai.response.finish_reasons: [stop] | [length] | [tool_calls]
gen_ai.usage.input_tokens: 1250 gen_ai.usage.output_tokens: 420 gen_ai.usage.total_tokens:
1670 # Standard OTel events within LLM spans: gen_ai.system.message: {role: system, content:
...} gen_ai.user.message: {role: user, content: ...} gen_ai.choice: {index: 0, finish_reason:
stop, message: ...} # Result: any OTel-instrumented AI app exports standardised # telemetry to
Tempo, Jaeger, Honeycomb, Datadog, or any backend
```

CHAPTER 5

Ecosystem Convergence and Adoption Recommendations

Standard / Tool	Maturity	Adopt Now	Watch Next 12 Months
MCP (Streamable HTTP)	GA production	All new tool integrations	MCP Server Operator CRD
A2A Protocol	Early production	New multi-agent system design	A2A discovery standards, CNCF proposal

Standard / Tool	Maturity	Adopt Now	Watch Next 12 Months
OTel GenAI conventions	GA stable	All LLM call instrumentation	OTel AI metrics working group output
OpenAI API compatibility	De facto standard	All inference deployments	Unified inference standard (IETF?)
AI-BOM / Model Card	Draft / Emerging	Pilot for new models	ISO/IEC 5338 ratification (2026)
SLSA for AI builds	Level 1-2 achievable	CI pipeline provenance	Level 3 for training pipeline
Confidential Computing	Early enterprise	Pilot for regulated AI	AMD SEV-SNP standard in managed K8s
Agent Mesh (A2A + service mesh)	Experimental	Architecture design	Standardisation in CNCF Agent WG